

Weighted Fair Queueing (WFQ)

- Round robin with different weights or priorities to processes
 - *Decided by scheduler or can be set by users*
 - *Time slice will be in proportion to the weight or priority*
- Real life schedulers may not be able to enforce time slice exactly
 - *What if timer interrupt is not exactly aligned with time slice?*
 - *What if process blocks before its time slice?*
- Practical modification: keep track of run time of process, schedule process that has used least fraction of its fair share
 - *Compensate excess/deficit running time in future time slices*
- Linux scheduler is a variant of weighted fair queueing
 - *CFS (completely fair scheduler) uses red-black trees to keep track of the fair share run time of processes, schedules the one with least run time*

Multi-level feedback queue (MLFQ)

- Linux uses a Multi Level Feedback Queue (MLFQ)
- Real schedulers are more complex
 - *Many queues, in order of priority*
 - *Process from highest priority queue scheduled first*
 - *Within same priority, any algorithm like RR*
 - *Priority of process decays with its age*

MLFQ Overview

- Multiple queues, one for each priority level
 - Schedule processes from highest priority queue to lowest
 - Use round robin scheduling for processes within same priority level

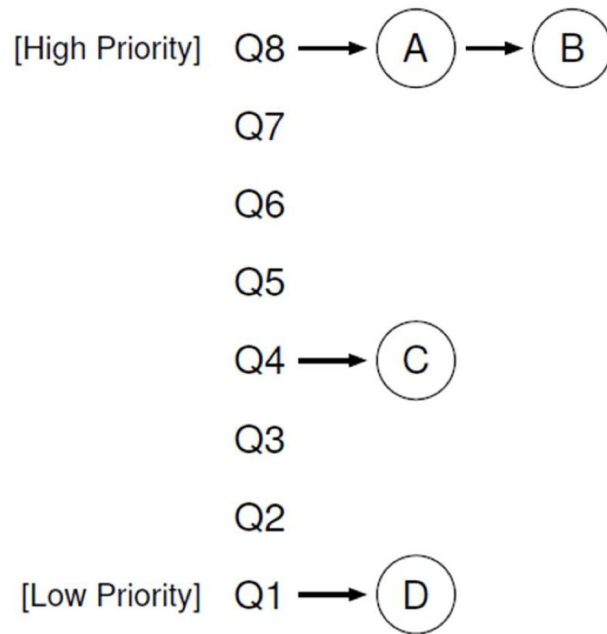


Figure 8.1: MLFQ Example

MLFQ Summary

- **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't).
- **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR.
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue).
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).
- **Rule 5:** After some time period S , move all the jobs in the system to the topmost queue.

MLFQ: I/O & CPU bound processes

- Prioritize short, I/O bound processes over long, CPU bound processes
 - *Why? System works best when short processes don't have to wait for too long behind longer processes (that's why SJF is optimal!)*
 - *How is this done? I/O bound processes will stay at highest priority level*
 - *We do this without knowing the run times apriori, so practical to work in real life scenarios*
 - *Real life schedulers cannot make assumptions on knowing run times etc, but still try to achieve good properties of theoretically optimal schedulers*

GATE CSE 2026 Set 2

GATE CSE 2026 Set 2 — 1 Mark

Which one of the following CPU scheduling algorithms cannot be preemptive?

- A. Shortest Remaining Time First (SRTF) Scheduling
- B. First Come First Serve (FCFS) Scheduling
- C. Round Robin Scheduling
- D. Priority Scheduling

GATE CSE 2026 Set 2

GATE CSE 2026 Set 2 — 1 Mark

Which one of the following CPU scheduling algorithms cannot be preemptive?

- A. Shortest Remaining Time First (SRTF) Scheduling
- B. First Come First Serve (FCFS) Scheduling
- C. Round Robin Scheduling
- D. Priority Scheduling

Answer: B — FCFS

GATE CSE 2002

GATE CSE 2002 — 1 Mark

Which of the following scheduling algorithms is non-preemptive?

- A. Round Robin
- B. First-In First-Out
- C. Multilevel Queue Scheduling
- D. Multilevel Queue Scheduling with Feedback

GATE CSE 2002

GATE CSE 2002 — 1 Mark

Which of the following scheduling algorithms is non-preemptive?

- A. Round Robin
- B. First-In First-Out
- C. Multilevel Queue Scheduling
- D. Multilevel Queue Scheduling with Feedback

Answer: B — First-In First-Out (FIFO/FCFS)

Summary

1. **Scheduling Policies** can be preemptive and non-preemptive
 - a. *Scheduling policy = decision engine = which one to run first*
2. **Mechanisms** are context switches, timer interrupts, preemption, dispatcher
 - a. *Mechanisms = machinery or operations to run that process*
3. **Real OS Scheduling policy** is far more complicated than standard scheduling policies
 - a. *Real OS use multiple scheduling policies based on nature of task*
4. Preemptive and Non-Preemptive Scheduling is **Classification of Scheduling Algorithms**
 - a. *Preemptive*
 - b. *Non-Preemptive*
5. Why do we need Scheduler?
6. Some Scheduling algorithms have both preemptive and non-preemptive variants.

Summary

Scheduling algorithm	Non-preemptive	Preemptive	
FCFS	Yes	—	—
SJF	Yes	—	—
SRTF	—	Yes	Yes
Priority	Yes	Yes	Yes
Round Robin	—	Yes	Yes
Multilevel Queue	Depends	Depends	
Multilevel Feedback Queue	Typically preemptive	Yes	